

Traitement automatique du langage: TAL

- Similarité du TAL
- Transformation des données
- Les modèles pré-entraînés
- Réduction de dimensionalité
- Exemples

Similarité pour TAL

Le TAL une tâche difficile

- *Le langage est complexe et structuré en plusieurs niveaux.*
- **Niveau phonologique** : Étude des sons de la langue (phonèmes) et de leur organisation (reconnaissance vocale, correction phonétique).
- **Niveau morphologique** : Structure interne des mots par l'identification des morphèmes (Lemmatisation : manges → manger, analyse des flexions (chanterai, chanté, chantant)).
- **Niveau syntaxique** : Organisation des mots en phrases. (Analyse syntaxique : parsing, détection de structures grammaticales).
- **Niveau sémantique** : Étude du sens des mots et des phrases. (Désambiguïsation lexicale, recherche d'information, traduction).
- **Contexte linguistique** : Le sens dépend aussi des phrases environnantes dans un texte. (Résolution de coréférence)
- **Niveau extra-linguistique** : Prise en compte la situation de communication (Chatbots, compréhension du contexte social).
- *Le TAL doit intégrer tous ces niveaux pour comprendre et produire un langage naturel de manière fiable.*

Similarité pour TAL

Principale étape du TAL

- *La première tâche concerne le nettoyage : selon les sources des données, cette phase consiste à réaliser des tâches telles que la suppression d'urls, d'emoji, etc.*
- *Tokenisation, ou découpage du texte en plusieurs parties : les tokens.*
- *Le stemming désigne le processus heuristique qui consiste à découper la fin des mots afin de ne conserver que la racine du mot.*
- *Un même mot peut se retrouver sous différentes formes : genre (masculin féminin) ; nombre (singulier, pluriel), personne (moi, toi, ...)*
- *Lemmatisation : qui consiste à réaliser la même tâche à partir un vocabulaire et une analyse de la construction des mots.*
- *La lemmatisation permet de supprimer par exemple les terminaisons inflexibles afin d'isoler la forme canonique du mot : lemme.*
- *d'autres opérations peuvent être nécessaires comme la suppression des chiffres, ponctuation, symboles et stopwords, passage en minuscule.*

Similarité pour TAL

La vectorisation des mots

- *Pour appliquer les méthodes de Machine Learning au langage naturel, il est indispensable de transformer les données textuelles en données numériques.*
- *Ces transformations doivent donner du sens au mots afin de pouvoir effectuer les différentes tâches comme le stemming, la lemmatisation ...*
- *Il existe plusieurs méthodes.*
- *Term-Frequency (TF) ou Term Frequency-Inverse Document Frequency (TF-IDF) : ces méthodes consistent à compter le nombre d'occurrences des tokens présents dans le corpus pour chaque texte.*
- *Inconvénient : La taille des vecteurs est liée à la taille du corpus, et le comptage des mots ne permet pas de rendre compte de leur agencement et donc du sens des phrases.*
- *Word Embedding. - consiste à construire des vecteurs de taille fixe qui prennent en compte le contexte dans lequel se trouvent les mots.*
- *Cela permet alors de capturer à la fois similarités sémantiques, syntaxiques ou thématiques des mots.*

Similarité en TAL

La similarité en Traitement Automatique du Langage

- *Le langage est riche et ambigu : deux textes peuvent avoir le même sens, et un même mot peut avoir plusieurs significations différentes.*
- *La similarité est fondamentale pour le traitement du langage, elle permet de mesurer à quel point deux textes se ressemblent.*
- *Elle permet également aux machines de comprendre, analyser et générer du langage humain.*
- *Comprendre et mesurer la proximité entre textes permet en outre : de rechercher des informations ; de faire des recommandations ; de traiter les plagats ou pour les chatbots : de comparer les requêtes à celles déjà traitées afin d'améliorer les réponses.*
- *La similarité ne se limite pas à une simple ressemblance entre mots : elle s'attache également à la proximité sémantique, c'est-à-dire au sens.*
- *Différentes approches ont été développées, allant des méthodes lexicales basées sur la comparaison de mots aux modèles vectoriels et contextuels capables de capturer les nuances du langage.*

Similarité en TAL

Similarité lexicale

- Les premières méthodes comparent les textes au niveau lexical, c'est-à-dire au niveau des mots.
- La distance d'édition (Levenshtein) correspond au nombre minimum d'opérations élémentaires (insertion, suppression, substitution) nécessaires pour transformer une chaîne en une autre.
- Elle est utilisée entre autres pour la correction orthographique, la recherche approximative dans des BdD textuelles, la détection de plagiat ou encore...

Par exemple, la distance d'édition entre les mots "DEN" et "SENT" est 3. On peut identifier les lettres qui sont identiques (D, E, N) et les lettres qui sont différentes (S, T).

La distance d'édition entre les mots "DEN" et "SENT" est 3. La distance d'édition entre les mots "DEN" et "SENT" est 3. La distance d'édition entre les mots "DEN" et "SENT" est 3.

PLS

	∅	D	E	T	E	N	T	E	S
∅	0	0	0	0	0	0	0	0	0
D	0	1	1	1	1	1	1	1	1
E	0	1	1	1	1	1	1	1	1
S	0	1	1	1	1	1	1	1	2
T	0	1	1	2	2	2	2	2	2
A	0	1	1	2	2	2	2	2	2
N	0	1	1	2	2	3	3	3	3
C	0	1	1	2	2	3	3	3	3
E	0	1	2	2	3	3	3	4	4

]]

Similarité en TAL

Similarité lexicale

- *L'indice de Jaccard, compare la proportion de mots partagés entre deux ensembles. Si A et B sont deux ensembles de mots :*

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

- *Plus la valeur est proche de 1, plus les ensembles sont similaires.*
- *Cette mesure est utilisée pour la comparaison de documents ou phrases par présence de mots, le filtrage de doublons dans des bases de données ou la recherche approximative dans des corpus textuels.*
- *Cette mesure ne permet pas de prendre pas en compte la fréquence des mots (un mot répété n'est pas plus important).*
- *Ne capture pas la sémantique : deux phrases synonymes mais avec mots différents auront J=0*
- *Pour une meilleure précision l'indice de Jaccard est combiné avec d'autres mesures (TF-IDF, embeddings).*

Similarité en TAL

Similarité lexicale

- La similarité cosinus est une mesure de proximité entre deux vecteurs.
- Elle mesure l'angle entre deux vecteurs, indépendamment de leur norme (longueur) seul compte la direction :

$$\text{cosinus_similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

- $A \cdot B$ correspond au produit scalaire entre les vecteurs A et B .
- $\|A\|$ et $\|B\|$ est la normes (longueurs) des vecteurs.
- Plus la valeur est proche de 1 et plus les vecteurs sont similaires, ils sont orthogonaux (aucune similarité) lorsque le cosinus est proche de 0.
- Cette mesure permet la comparaison de mots ou phrases représentés par des vecteurs (Word2Vec, GloVe, BERT)
- Elle permet de capturer la similitude sémantique et est facile à calculer sur des vecteurs numériques.
- Cette mesure dépend de la qualité des vecteurs (embeddings) et ne permet pas gérer le contexte complet lorsque les modèles sont simples.

Similarité en TAL

Les principales représentations vectorielles

- *TF-IDF (Term Frequency – Inverse Document Frequency).*
- *La représentation vectorielle dépend de la fréquence des mots dans le document (TF) et de leurs raretés le corpus (IDF).*
- *TF-IDF est simple mais ne permet pas de capturer le sens ni le contexte. De plus elle conduit à de grands vecteurs creux.*
- *Word2Vec (2013, Google) est une représentation liée à la distribution des mots dans le corpus. "Le sens d'un mot dépend de son contexte"*
- *Word2Vec permet de capturer la sémantique et de mieux comprendre les relations entre les mots. Les vecteurs sont denses et compacts.*
- *Cette représentation ne gère pas les mots rares ou inconnus (OOV), et n'est pas en mesure de prendre en compte le contexte global.*
- *Cette représentation ne sait donc pas faire la différence entre des mots similaires (polysémie). Exemple : Avocat.*
- *Glove (2014, Stanford University) combine les approches basées sur la prédiction (Word2Vec) et celles sur les cooccurrences globales (TF-IDF).*

Similarité en TAL

Les principales représentations vectorielles

- *FastText (2016, Facebook) est une évolution de modèles tels Word2Vec. Chaque mot est décomposé en n-grammes de caractères.*
- *Exemple (trigrammes) « Avocat » = "<Av"+ "Avo"+ "voc"+ "oca"+ "cat"+ "at>", sera représenté par la somme ou la moyenne des 3-grammes.*
- *Cette représentation est bien adaptée pour des langues riches morphologiquement comme le Français.*
- *Ainsi même les mots rare ou inconnus peuvent avoir une représentation car ils contiennent des n-grammes entraînés.*
- *Par contre comme pour Word2Vec, FastText produit des vecteurs statiques. Un mot = un vecteur, donc le sens ne dépend pas du contexte.*
- *Cette représentation ne gère pas la polysémie.*
- *FastText est alors souvent remplacé par BERT ou Sentence-BERT, qui produisent des représentations dynamiques dépendantes du contexte.*

Similarité en TAL

Les modèles contextuels

- Permettent de tenir compte des ambiguïtés lexicales (ex. "avocat" = personne ou fruit)
- Les modèles contextuels (ex : BERT, GPT, RoBERTa) représentent les mots en fonction de leur contexte dans la phrase.
- Contrairement aux modèles traditionnels (Word2Vec, GloVe) qui donnent un vecteur fixe par mot, ces modèles génèrent un vecteur différent selon la phrase où le mot apparaît.
- Par exemple pour le mot « avocat » BERT donnera deux vecteurs différents, reflétant le sens réel selon le contexte, alors que des modèles tels que (Word2Vec ou GloVe) auront le même vecteur.
- Ces modèles captent les nuances de sens et les paraphrases. Ils sont adaptés pour : la recherche d'information contextuelle, les Chatbots intelligents et la détection de doublons ou de plagiat sémantique.
- Par contre le coût computationnel est élevé et ils dépendent de la qualité du modèle pré-entraîné et du domaine des textes.

Transformation des données

Le langage a base de l'intelligence artificielle

- Le TAL (Natural Language Processing) est une technologie qui permet aux machines de comprendre le langage humain.
- Cette technologie est l'un des principaux outils à la base des modèles d'IA, puisqu'elle permet aux machines d'interagir avec les humains.
- Pour un RdN, il est indispensable de transformer les données brutes (texte) en données exploitables.
- Parmi ces étapes on retrouve une phase de nettoyage qui consiste à supprimer certains caractères, les emoji, les urls ...
- La tokenisation consiste ensuite à découper le texte en token. Un token peut représenter un caractère un mot ou un groupe de mots (n-gramme).
- Le stemming et la lemmatisation sont des techniques utilisées pour transformer les mots en leur radical ou à isoler leur forme canonique.
- Enfin, il peut être nécessaire d'effectuer d'autres opérations comme la suppression de chiffres, des ponctuations, des symboles ...

Transformation des données

Transformations des données textuelles

- Le one-hot-encoding consiste à associer à chaque token un vecteur de taille égale au nombre de tokens composé d'un seul un et de zéros.
- Term-Frequency (TF) consiste elle à compter les occurrences des tokens dans les corpus de chaque texte.
- La représentation est alors un vecteur d'occurrence (Bag-Of-Word).
- Term-Frequency Inverse Document Frequency (TF-IDF) est une mesure qui permet d'évaluer l'importance d'un token, relativement à un corpus.
- Par exemple un token qui apparait dans toutes les phrases peut avoir moins d'importance qu'un mot qui apparait moins souvent.
- Ces deux méthodes ne sont pas en mesure de prendre en compte l'agencement des mots, c'est-à-dire le sens des phrases.
- Le Word Embedding consiste à construire des vecteurs qui prennent en compte le contexte dans lequel se trouvent les tokens.
- Les tokens utilisés dans des contextes similaires seront proches, ce qui permet de capturer leurs similarités sémantiques ou thématiques.

TAL : one hot encoding

Représentation des mots

- Afin de pouvoir être utilisé par des méthodes d'apprentissage les mots doivent donc être représentés (encodés) grâce à des valeurs numériques.
- Les mots peuvent être encodés par un nombre représentant leur position dans le vocabulaire des mots ou par des vecteurs.
- La représentation vectorielle est bien plus porteuse de sens qu'un encodage lié à la position du mot dans le vocabulaire.
- La représentation vectorielle la plus classique est le one hot encoding.
- Chaque mot est représenté comme un vecteur binaire avec toutes ses valeurs nulles à l'exception de la position du mot dans le vocabulaire.

$V = \{ \text{'chasse'} : 0, \text{'chat'} : 1, \text{'chien'} : 2, \text{'devore'} : 3, \text{'fromage'} : 4, \text{'la'} : 5, \text{'le'} : 6, \text{'mange'} : 7, \text{'souris'} : 8 \}$

chasse = [1, 0, 0, 0, 0, 0, 0, 0, 0]
chat = [0, 1, 0, 0, 0, 0, 0, 0, 0]
chien = [0, 0, 1, 0, 0, 0, 0, 0, 0]
...
souris = [0, 0, 0, 0, 0, 0, 0, 0, 1]

TAL : one hot encoding

Inconvénients du one-hot

- La taille d'un vecteur pour un mot dépend de la taille du vocabulaire.
- En fonction du vocabulaire le one-hot encoding crée un grand nombre de dimensions, ce qui rend les modèles moins efficaces.
- Pour des tokens 'caractères' ou pour certains modèles de classification le one-hot peut être utilisé, mais sinon il présente trop d'inconvénients.
- De plus le one-hot-encoding ne contient pas d'informations sur la sémantique ou même le contexte d'un mot.
- A partir d'une représentation de type one-hot il est difficile voir impossible pour un modèle de pouvoir comprendre le sens d'une phrase.
- Pour appréhender le sens d'une phrase, l'encodage doit permettre de représenter les mots mais également de leur donner un sens.

Le chat chasse la ????

Le chat mange le ????

Le chien chasse le ????

Le ???? chasse la souris

TAL : word embedding

Représentation des mots avec TF-IDF

- Pour comprendre le sens des phrases, la première étape consiste à les représenter par des vecteurs.
- tf-idf "Term Frequency-Inverse Document Frequency" permet d'encoder une phrase sur un vecteur de taille égale au nombre de mots du corpus.
- Un mot qui apparaît souvent dans la phrase sans apparaître dans les autres aura un score tf-idf plus élevé.
- A l'inverse un mot très fréquent dans tout le corpus sera sous noté.

#Modèle de calcul du tf-idf

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
corpus = ["le chat mange la souris", "le chien chasse le chat", "la souris devore le fromage"]
```

```
tfidf = TfidfVectorizer()
```

```
corpus_vector = tfidf.fit_transform(corpus); index=[f"phrase {i+1}" for i in range(len(corpus))]
```

```
pd.DataFrame(corpus_vector.toarray(), columns= tfidf.get_feature_names_out(), index=index)
```

Avec

N le nombre de phrases

m le mot à encoder
d la phrase traitée

D l'ensemble des phrases du corpus

tf(m,d) occurrence de m dans d

df(m) nombre de phrases d ∈ D qui contiennent le mot m

$$tfidf(m, d) = tf(m, d) \times idf(m, D)$$

$$idf(m, D) = \ln \left(\frac{1 + N}{1 + df(m)} \right) + 1$$

	chasse	chat	chien	devore	fromage	la	le	mange	souris
phrase 1	0,00	0,43	0,00	0,00	0,00	0,43	0,34	0,57	0,43
phrase 2	0,50	0,38	0,50	0,00	0,00	0,00	0,59	0,00	0,00
phrase 3	0,00	0,00	0,00	0,53	0,53	0,41	0,32	0,00	0,41

La méthode **TfidfVectorizer()** de sklearn permet cet encodage et normalise ensuite les valeurs $idf(m, D)$ à un à partir de la somme des carrés des tokens de la phrase.

TAL : word embedding

La sémantique des mots avec TD-IDF

- Comme les phrases sont représentées par des vecteurs il est possible de mesurer les distances qui les séparent, c'est-à-dire leur ressemblance.
- Cette métrique peut s'appliquer pour mesurer la distance sémantique entre les mots.
- On utilise l'hypothèse selon laquelle les mots qui apparaissent dans les mêmes contextes ont des significations similaires.
- Il nous faut créer un vecteur correspondant au contexte des mots, c'est-à-dire identifier les mots qui se trouvent dans son voisinage immédiat.

#Corpus

```
corpus=["le chat mange la souris",  
        "le chien chasse le chat",  
        "la souris devore le fromage",  
        "le chien aime des croquettes",  
        "le chat chasse la souris",  
        "le chat aime le fromage",  
        "le chat mange des croquettes"]  
contextes = {}
```

#Identification des mots appartenant au contexte d'un mot

```
for phrase in corpus:  
    words = phrase.split()  
    for i, word in enumerate(words):  
        index = [(i+j) for j in range(-2,3) if (j!=0)]  
        select = [j for j in index if (j>=0) and (j<len(words))]  
        if word in contextes:  
            contextes[word]=contextes[word]+[words[j] for j in select]  
        else : contextes[word]=[words[j] for j in select]
```

TAL : word embedding

La sémantique des mots avec TD-IDF

- A partir du contexte des différents mots il est possible de créer un vecteur correspondant à l'encodage des contextes.
- On utilise pour cela le modèle tf-idf entraîné sur le corpus.

#Identification des mots appartenant au contexte d'un mot

tfidf = TfidfVectorizer()

#Corpus des mots appartenant au contexte d'un mot

def find_context(words, word):

scores = {}
for k in words:
scores[k] = ' '.join(v) ; words_vector[k] = tfidf.transform([scores[k]]).toarray()[0]

words = [k for k in contexte.keys() if k != word]

#Affichage des vecteurs des mots

def plot_vector(word):
scores = cosine(words_vector[word],
plt.figure(figsize=(20,1))
words_vector[word])

df = pd.DataFrame(words_vector[word], index=[word], columns=list(contexte.keys()))

sns.heatmap(df, np.around(min(scores),3)

#Affichage des similarités entre les mots

for k in contexte.keys():

print(f'{k:15} ==> {most_similar(k)}')

plot_vector('chat')

la ==> ('la', 0.447)
chat ==> ('chien', 0.206)
mange ==> ('mange', 0.258)
la ==> ('le', 0.447)
souris ==> ('chat', 0.342)
chien ==> ('chat', 0.206)
chasse ==> ('mange', 0.258)
devore ==> ('chasse', 0.466)
fromage ==> ('chien', 0.41)
aime ==> ('mange', 0.321)
des ==> ('le', 0.466)
croquettes ==> ('chien', 0.468)

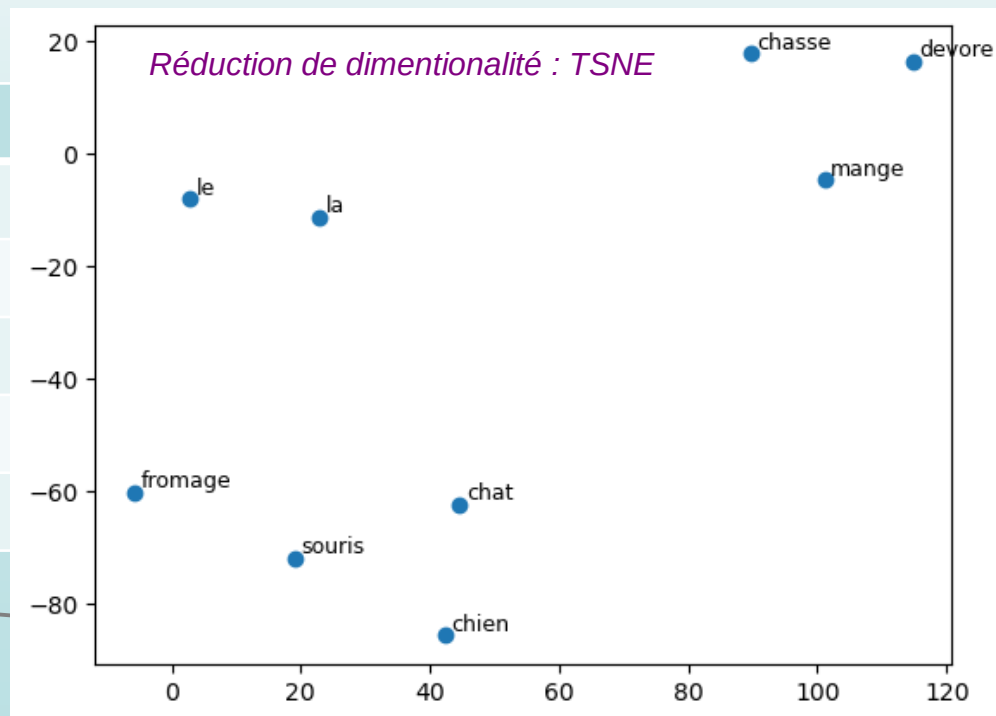


TAL : word embedding

Principes de word embedding

- L'objectif du word embedding est de trouver une représentation vectorielle qui donne du sens aux différents mots.
- Le principe consiste à encoder les mots du vocabulaire en fonction d'un certain nombre de caractéristiques.

	chasse
Animal	0.06
Action	0.96
Nourriture	0.02
Conjonction	0.02
Genre	0.02



mange	souris
0.05	0.97
0.99	0.04
0.08	0.54
0.02	0.01
0.02	-0.98

rielle

TAL : word embedding

Représentation des mots : pré-entraîné

- A la base de tout modèle TAL il y a donc la représentation de mots.
- Le word embedding est un ensemble de techniques permettant de transformer un mot ou ensemble de mots sous la forme de vecteurs.
- L'idée consiste alors non seulement à coder les mots mais surtout de trouver une représentation qui permet de donner un sens aux mots.
- Par exemple la technique Word2Vec (algorithme de word embedding) développée par google permet de donner un sens aux différents mots.
- En donnant du sens aux mots en fonction du contexte les algorithmes peuvent comprendre et interpréter nos requêtes.
- Gensim est une bibliothèque de TAL d'apprentissage non supervisée, utilisée pour le pré-traitement des textes

#Chargement d'un modèle pré-entraîné

```
import gensim.downloader as api  
model = api.load('word2vec-google-news-300')  
model.most_similar(positive=['king', 'woman'],  
negative=['man'], topn=1)
```

```
[('queen', 0.7118193507194519)] #Out  
model.most_similar(positive=['puppy', 'cat'],  
negative=['dog'], topn=1)  
[('kitten', 0.7634989619255066)] #Out
```

TAL : word embedding

Représentation des mots

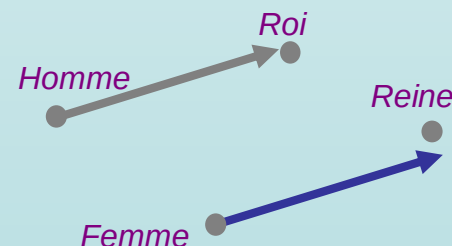
- Dans un espace mathématique les mots sont représentés par des points dans un espace vectoriel de dimension N .
- Il est possible de calculer des distances entre les mots.
- Il est également possible d'appliquer des transformations qui permettent de "calculer" les relations sémantiques qui lient les mots.

$$\text{Homme} - \text{Femme} \approx [-2, 0, 0, 0, 0]$$

$$\text{Roi} - \text{Reine} \approx [-2, 0, 0, 0, 0]$$

$$\text{Femme} + \overrightarrow{\text{Roi} - \text{Homme}} \approx \text{Reine}$$

	Homme	Femme	Roi	Reine
Sexe	-1	1	-0.99	0.99
Rang	0.17	0.15	1	0.98
Nourriture	0.02	0.03	0.01	0.01
Conjonction	0.02	0.01	0.01	0.02
Responsabilité	0.22	0.2	0.95	0.92



$F + \overrightarrow{R - H}$
1.01
0.98
0.03
0.00
0.93

TAL : Exemple de pré-embedding

Tokenisation

- La tokenisation qui consiste à découper un texte en tokens, généralement des mots, peut être liée à des entités plus petites.
- Les séquences à exploiter (phrases) sont alors segmenter en tokens (valeurs numériques) exploitables par les modèles d'apprentissage.
- La tokenisation n'est pas en mesure de traiter les variations morphologiques (ex : parler et parle sont des tokens différents).
- Pour traiter les variations morphologiques, les modèles découpent les mots en sous-mots (ex : préfixe + radical + suffixe, syllabes ...).
- Principales méthodes de découpage en sous-mots : Byte Pair Encoding (GPT), WordPiece Tokenization (BERT), Unigram Language Model

#Exemple de traitement de fake news

fake = pd.read_csv('Fake.csv')

#Pré-traitement des données

def : text_preprocessing (x) :

#majuscules -> minuscules ; Suppression des

caractères spéciaux, ponctuations, etc ...

return x

#Pré-traitement des textes, et mise en forme

*fake_news['text']=fake_news['text'].apply(lambda
x : text_preprocessing(x))*

print(fake.iloc[0]['text'])

"donald trump wish americans happy new year

leave instead give shout enemies haters

dishonest fake news media former reality show..."

TAL : Exemple de pré-embedding

Tokenisation

- Keras propose un outil de tokenisation permettant de transformer du texte en séquences numériques exploitables.
- A la création du "tokenizer" il est possible de limiter la tokenisation a un nombre limite de mots, d'ignorer certain caractères (ponctuation), ou de remplacer les mots hors vocabulaire par un token spécifique.

```
#Création d'une liste texte. Chaque élément de la liste est une liste de mots. Tenseur de dim=2
X = [x.split() for x in fake['text'].tolist()]
#Création d'un modèle de tokenisation limité aux 10000 mots les plus fréquents
from keras_preprocessing.text import Tokenizer
tokenizer = Tokenizer(num_words=10000, filter='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~\t\n', oov_token="<OOV>")
tokenizer.fit_on_texts(X)
#Apprentissage du vocabulaire sur les fakes
```

```
#Affichage des premiers tokens
for word, index in tokenizer.word_index.items():
    print(word, ' : ', index)

trump : 1      would : 5      new : 9
said : 2       people : 6     obama : 10
u : 3          one : 7        also : 11
president : 4  state : 8      clinton : 12. ....
```

```
#Tokenisation des fakes
X=tokenizer.texts_to_sequences(X)
#Affichage de la première fake tokenisée
[ 14 1 2366 118 1547 9 18 551 426 310 7707 3073
 8152 4680 510 28 38 .... ]
#Création de séquence de même longueur
line_size=1000 ; X=pad_sequences(X,line_size)
```


TAL : Exemple de pré-embedding

Embedding

- Afin de pouvoir utiliser un modèle pré-entraîné, il faut dans un premier temps transformer les mots du texte en vecteurs de sens.
- Il faut ensuite créer un dictionnaire liant tous les mots du texte et leur vecteur dense associé.
- Ce dictionnaire pourra ensuite être fourni à une couche d'embedding.
- Comme les mots sont pré-entraînés il n'est donc pas nécessaire de mettre à jour les poids des neurones de la couche d'embedding.
- L'apprentissage sera plus rapide, mais moins bien adapté au problème.

```
import gensim                                     #Utilisation de l'agorithme Word2Vec de gensim
w2v=gensim.models.Word2Vec(sentences=X, vector_size=100, window=10, min_count=1)
w2v.wv.most_similar(['france'])                  #Affichage des mots liés
#[ ('french', 0.6893...), ('italy', 0.6775...), ('finland', 0.6657...), ('germany', 0.6583...), ....]
#Création d'une matrice d'embedding de
vocab_size=len(tokenizer.word_index)+1 ; Dim=100
weight_of_word = np.zeros((vocab_size,Dim))
for word, index in tokenizer.word_index.items() :
    weight_of_word[index]=w2v.wv[word]
```

TAL : Exemple de pré-embedding

Modèle d'apprentissage

- Un embedding sur les "True" news doit être effectué afin de disposer d'un jeu de données complet.
- Un modèle complet pour la classification des news peut utiliser une couche de type LSTM, normale ou bidirectionnelle.
- Une classification issue d'une couche Dense est la dernière couche du modèle.

```
model = Sequential()
#Les données dépendent un vecteur de taille 1000 (d'une seule dimension).
model.add(tf.keras.layers.Input(shape=(line_size,)))
#C'est cette couche qui convertit des entiers (index de mots) en vecteurs denses de taille fixe.
#vocab_size : définit la taille du vocabulaire ; Dim : la taille des vecteurs d'embedding
#Weights : Charge la matrice des poids pré-entraînée pour l'embedding (Word2Vec, ou GloVe).
#trainable : les poids de l'embedding ne seront pas mis à jour pendant l'entraînement
model.add(keras.layers.Embedding(input_dim=vocab_size, output_dim=Dim,
                                weights=[embedding_vector], trainable=False))
model.add(keras.layers.LSTM(units=128))
model.add(keras.layers.Dense(1, activation='sigmoid'))
```

TAL : Le modèle Spacy

Compréhension et analyse du langage naturel

- Plusieurs bibliothèques fournissent des outils d'analyse du langage, la plupart sont entraînées sur des corpus anglais.
- Néanmoins il existe des librairies de traitement du langage avec des modèles pré-entraînés sur des corpus français (SpaCy, NLTK ...).
- SpaCy est conçue pour prendre en charge des modèles personnalisés sous TensorFlow ou PyTorch.
- NLTK est plutôt utilisé pour l'enseignement et la recherche.
- Ces bibliothèques disposent de fonctionnalités permettant notamment l'analyse de sentiments, de relations, l'identification d'entités nommées...
- La première étape consiste à télécharger le modèle à utiliser, pour le français il existe quatre modèles : 'fr_core_news_' + {'sm', 'md', 'lg', 'trf'}.
- Il est ensuite possible de traiter un corpus pour le NLP.

```
import spacy
nlp = spacy.load("fr_core_news_sm")           #Chargement d'un français 'small'
doc = nlp(" Le 24 février 1815, la vigie de Notre-Dame de la Garde signala le trois-  
mâts le Pharaon, venant de Smyrne, Trieste et Naple.")
```

TAL : Le modèle Spacy

Tokenisation : première étape

- La tokenisation prépare le texte pour des tâches telles que la classification, l'analyse des sentiments, l'extraction d'informations ...
- Ce découpage permet de représenter le texte sous une forme qui puisse alors être traitée par les différentes fonction de la librairie.
- La variable doc est une liste de tokens qui sont soit des mots, soit des symboles de ponctuation, ou tout autre élément constituant un texte.

```
print(type(doc))           # spacy.tokens.doc.Doc  
print(type(doc[0]))        # spacy.tokens.token.Token
```

- La variable doc contient 26 tokens dont 4 sont associés à des symboles de ponctuation (3 virgules et 1 point) et 22 sont à des mots.

```
print( [token.text for token in doc]  
#output : ['Le', '24', 'février', ... , ',', 'la', 'vigie', ... 'trois-mâts', ... , 'Naple', '.']
```

- Lors de la phase de tokenisation sur le corpus, la fonction nlp identifie également les phrases que l'on peut récupérer à partir de doc.sents.

TAL : Le modèle Spacy

Etiquetage morpho-syntaxique (Part-Of-Speech tagging)

- Le Part-Of-Speech consiste à associer aux tokens, en fonction de leur contexte, des informations grammaticales correspondantes.
- Ces informations sont accessibles sous forme d'attributs : la classe (pos_), la forme de base "dictionnaire" (lemma_) le genre, le nombre.

```
doc=nlp("Le 24 février, les vigies signalèrent le trois-mats venant de Naples.")
for token in doc:
    print(f"{token.text:<10} | {token.pos_:<10} | {token.lemma_:<10} | {str(token.morph):<10}")
```

#Output

Le	DET	le	Definite=Def Gender=Masc Number=Sing PronType=Art
24	NUM	24	NumType=Card
février	NOUN	février	Gender=Masc Number=Sing
,	PUNCT	,	
les	DET	le	Definite=Def Number=Plur PronType=Art
vigies	NOUN	vigie	Gender=Fem Number=Plur
signalèrent	VERB	signaler	Mood=Ind Number=Plur Person=3 Tense=Pres VerbForm=Fin
le	DET	le	Definite=Def Gender=Masc Number=Sing PronType=Art
trois-mats	NOUN	trois-mat	Gender=Masc Number=Sing
venant	VERB	venir	Tense=Pres VerbForm=Part
de	ADP	de	
Naples	PROPN	Naples	
.	PUNCT	.	

TAL : Le modèle Spacy

Etiquetage morpho-syntaxique

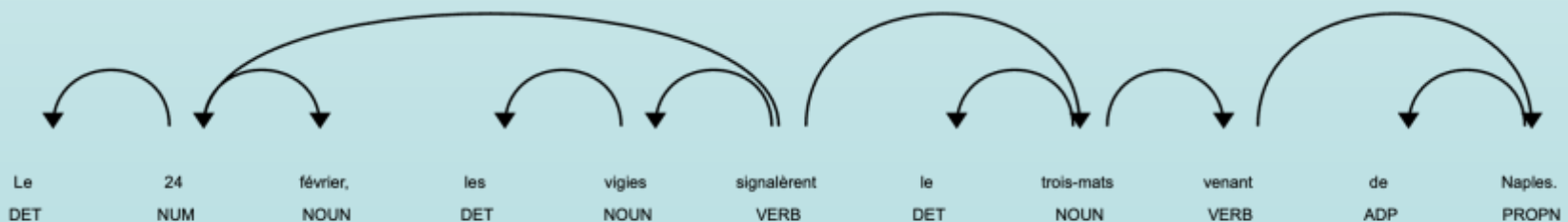
- L'étiquetage permet également de construire un arbre de dépendance basé sur les liens existants entre les tokens de la phrase.
- Les dépendances grammaticales (dep_), permettent de construire l'arbre.

for token in doc:

```
print(f'({token.text} : {token.dep_})", end=' ; ')
```

```
(Le : det) ; (24 : obl:mod) ; (février : nmod) ; (, : punct) ; (les : det) ; (vigies : nsubj) ; (signalèrent : ROOT) ; (le : det) ; (trois-mats : nsubj) ; (venant : acl) ; (de : case) ; (Naples : obl:arg) ; (. : punct) ;
```

```
from spacy import displacy
displacy.render(doc)
```



TAL : Le modèle Spacy

Les stop-words

- Les mots courant d'une langue n'apportent pas d'information.
- Pour certaines applications (classification ou l'analyse de sentiments), ces mots n'apportent pas d'information et peuvent être supprimés.
- En les identifiants et en les supprimant les TAL peuvent se concentrer sur les mots importants.
- Les stop words sont des listes disponibles dans les librairies.

(spacy) `spacy.lang.fr.stop_words.STOP_WORDS`

ou (nltk) `nltk.corpus.stopwords.words('french')`

- Il est possible de vérifier si un token est un stop word soit en vérifiant qu'il appartient à la liste des `stop_words`, soit en utilisant l'attribut `is_stop`.

```
[token.text for token in doc if token.is_stop]
```

```
#output : ['Le', 'les', 'le', 'de']
```

#9 mots sont supprimés

TAL : Le modèle Spacy

Reconnaissance d'entités nommées (NER)

- La NER consiste à identifier et catégoriser des tokens dans le texte.
- Les catégories génériques peuvent identifier une personne, une organisation, un lieu ...
- L'attribut `doc.ents` identifie les tokens appartenant à ces catégories, l'attribut `label_` de ces tokens contient leur catégorie.

```
doc=nlp("Un grand malheur, monsieur Morrel répondit le jeune homme, un grand malheur, pour moi surtout : à la hauteur de Civita-Vecchia, nous avons perdu ce brave capitaine Leclère.")  
[(token.text,token.label_) for token in doc.ents]  
#output : [('Morrel', 'PER'), ('Civita-Vecchia', 'LOC'), ('Leclère', 'PER')]
```

#Affichage du texte avec annotation des entités nommées

```
spacy.displacy.render(doc1, style='ent')
```

Un grand malheur, monsieur **Morrel PER** répondit le jeune homme, un grand malheur, pour moi surtout : à la hauteur de **Civita-Vecchia LOC**, nous avons perdu ce brave capitaine **Leclère PER**.

- La NER est essentielle pour extraire des informations avec précision.
- Une approche classique de construction consiste à utiliser un modèle Bidirectional Encoder Representations from Transformers (BERT).

TAL : Le modèle Spacy

Lemmatisation des tokens

- La lemmatisation consiste à ramener le mot à sa forme de base (tel qu'on le retrouve dans le dictionnaire).
- Grâce à la lemmatisation toutes les déclinaisons d'un mot peuvent être traitées de manière identique.
- La lemmatisation améliore les recherches, limite la taille du vocabulaire et permet d'améliorer les modèles du NLP.

```
# Récupération des différents mots du doc
mots = set()
for token in doc:
    # On ne traite que les mots qui ont du sens
    if (token.is_punct) or (token.is_stop) or
       (token.like_num) or
       (token.lemma_.lower() in mots) :
        continue
    if (token.has_vector):
        mots.add(token.lemma_.lower())
```

```
# Recherche de l'occurrence des mots
from collections import Counter
compteur = Counter(mots)
top_mots = compteur.most_common(20)
for mot, freq in top_mots:
    print(f"{mot} : {freq}")
dantès : 149
danglars : 131
fernand : 111
....
```

TAL : Le modèle Spacy

Similarité cosinus

- La similarité cosinus mesure le degré de similitude des documents.
- Elle est indépendante de la taille (distance euclidienne). La présence de plusieurs occurrences d'un mot ne change pas son orientation.
- Plus l'angle est petit, plus la similarité entre les tokens est importante.

$$\cos(\theta) = \frac{\vec{A} \times \vec{B}}{\|\vec{A}\| \times \|\vec{B}\|} \text{ avec } \vec{A} \text{ et } \vec{B} \text{ représentation vectorielle des tokens}$$

- La fonction `token1.similarity(token2)` permet de calculer la similarité entre deux tokens.
- Il est toute fois plus efficace d'utiliser un modèle de type transformer.

Récupération des différents mots du doc

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
emb1 = model.encode(phrase1, convert_to_tensor=True)
emb2 = model.encode(phrase2, convert_to_tensor=True)
util.cos_sim(emb1, emb2)
```

TAL : Le modèle Spacy

Exemple Question-réponse

- L'analyse de similarité peut permettre de répondre à des questions.

```
# SentenceTransformer classe qui permet de faire de la recherche sémantique, du clustering, ou du
calcul de similarité, sur la base de modèles pré-entraînés. Util classe de manipulation des embedding
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer("all-MiniLM-L6-v2")
corpus = [ "Les chats aiment les croquettes.", "Les chiens ne mangent pas les souris.", "Les souris
          n'aiment pas les croquettes.", "Les chiens aboient la nuit.", "Les chats chassent parfois les souris." ]
# Tokenisation du corpus
corpus_embeddings = model.encode(corpus, convert_to_tensor=True)
```

```
# Tokenisation de la requête
query_embedding = model.encode("Qui aime les croquettes ?", convert_to_tensor=True)
# Calcul de similarité entre la requête et les phrases du corpus
scores = util.cos_sim(query_embedding, corpus_embeddings)
scores[0]          #tensor([0.8586, 0.5001, 0.8482, 0.5261, 0.5495], device='mps:0')
```

Réduction de dimensionnalité

Le fléau de la dimension

- Le fléau de la dimension désigne les phénomènes qui adviennent lorsque l'espace des données est de grande dimension.
- L'espace croît exponentiellement avec les dimensions, les données se retrouvent alors "isolées", et il n'est plus possible de tirer des inférences.
- Avec un corpus de centaines de milliers de mots, analysé à partir de plusieurs millions de textes les modèles ne peuvent plus être entraînés.
- La solution consiste à représenter les mots dans un espace de plus petite dimension, tout en conservant les informations essentielles.
- Deux types de méthodes en machine learning permettent de réduire la dimensionnalité. Les méthodes linéaires et non linéaires
- Les méthodes linéaires (PCA, SVD ou LDA) ne conservent pas les distances, ainsi la distance entre les mots varie d'un espace à l'autre.
- Si la distance varie la similarité entre les mots est alors modifiée.
- On préfère alors utiliser des méthodes telles que t-SNE, Isomap ou LLE.

Réduction de dimensionnalité

LLE : Locally Linear Embedding

- LLE est une approche de réduction de dimensionnalité qui repose sur l'idée de préserver la structure locale des données.
- LLE est particulièrement bénéfique lorsque les données ont des structures de données non linéaires.
- LLE assez similaire à la méthode Isomap, se décompose en 3 étapes.
- 1- K-NN permet de trouver les k voisins les plus proches des points.
- 2 - Une matrice de poids est calculée afin de minimiser la fonction :

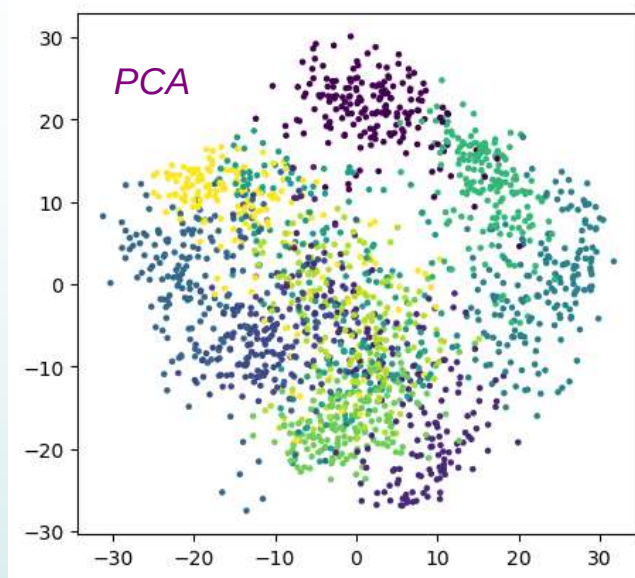
$$E(w) = \sum_i \left| X_i - \sum_j w_{i,j} X_j \right|^2 \quad \text{Avec} \quad \sum_j w_{i,j} = 1$$

- 3 - Trouver les vecteurs y_i correspondant à la projection des points x_i dans l'espace réduit en minimisant la fonction de coût.

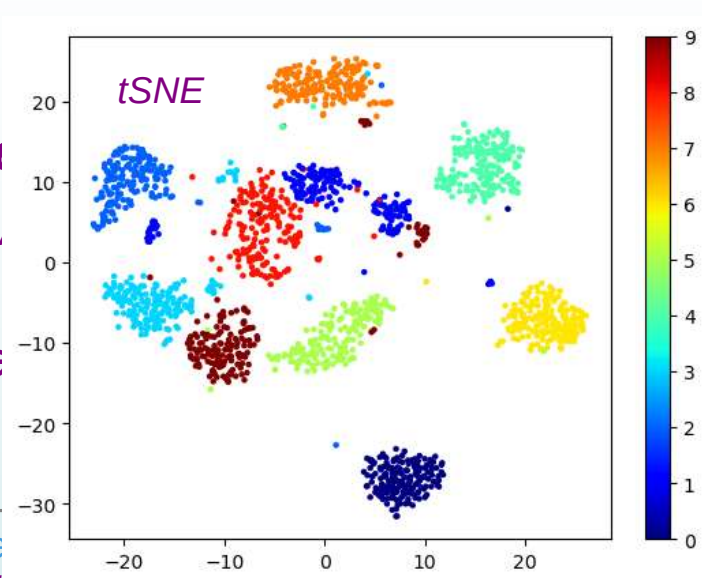
$$E(w) = \sum_i \left| Y_i - \sum_j w_{i,j} Y_j \right|^2$$

Traitement automatique du langage

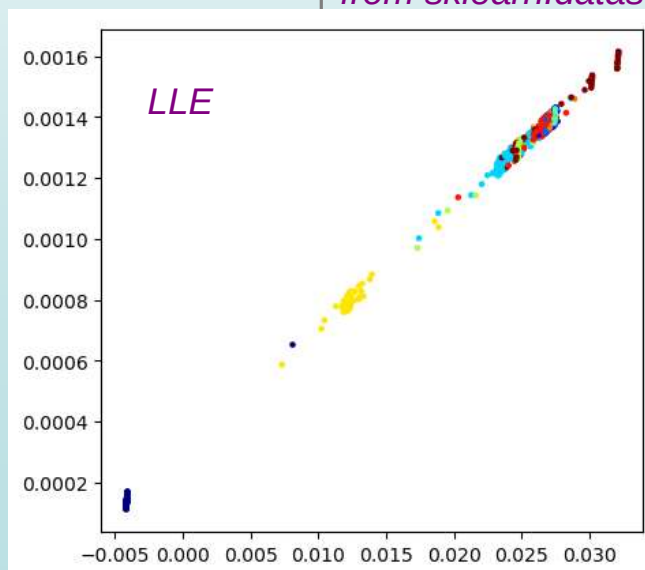
Master 2



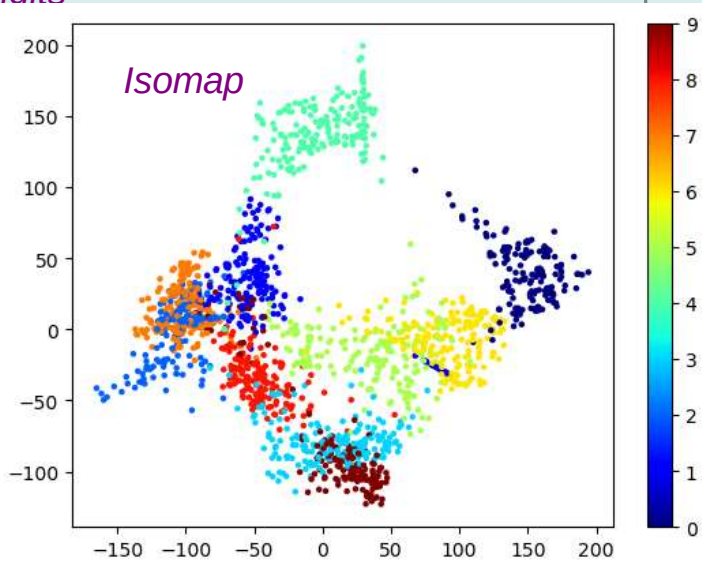
des m
E conserv
es dans
s dans l'e
que l'on
des donnée



nalité
oisines
L.



ion import
port TSNE,
= digits.data
= acp.fit_tra
_rate='auto
onents=2,
bors=5); X



PCA
tSNE
LLE
Isomap

Exemples

Analyse des critiques de films

- Keras fournis un jeu de données IMDB de 50000 critiques de films classés (1=positif/0=négatif).
- 25000 critiques servent pour l'apprentissage et 25000 pour le test. Il y a autant d'avis positifs que négatifs.
- Les tokens (mots) des critiques sont encodées par rapport à leur occurrence dans l'ensemble des critiques (format type TF).
- Le classement des critiques se fait sur la présence ou l'absence des mots et non pas sur le sens des phrases.

```
from keras.datasets import imdb                                #Téléchargement des données
num_words=10000                                                #Les mots rares sont écartés
(x_train, y_train), (x_test, y_test)= imdb.load_data(num_words=num_words)
words_to_index = imdb.get_word_index()
#Création des dictionnaires de correspondance entre les mots et leur représentation
words_to_index = {w:(i+3) for w, i in words_to_index.items()}
#Ajout des mots spécifiques correspondant aux tokens spécifiques
words_to_index.update({'<pad>':0, '<start>':1, '<unknown>':2, '<undef>':3,})
index_to_word = dict((i , word) for (word, i) in word_to_index.items())
```

Exemples

Analyse des critiques de films

- `x_train` contient les critiques des 25000 films sous la forme de tableaux de mots encodés en fonction de leur occurrence.

```
np.array(x_train[0])           #Affichage de la première critique  
array([ 1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, 458, 4468, 66, 3941, 4, 173, 36,  
256, 5, 25, ... 103, 32, 15, 16, 5345, 19, 178, 32])  
len(x_train[0])                #218 (nombre de mots de la critique)
```

```
" ".join(index_to_word[i] for i in x_train[0])  
'<start> this film was just brilliant casting location scenery story direction everyone's  
really suited the part they played and you could just imagine being there robert ...'
```

- Toutes les critiques ont des tailles différentes. Afin de pouvoir les traiter les tailles des critiques doivent être normalisées.

```
from tf.keras.preprocessing.sequence import pad_sequences  
review_len = 256                      #Taille des critiques (complété par des 0)  
x_train = pad_sequences(x_train, value=0, padding='post', maxlen=review_len)  
x_test = pad_sequences(x_test, value=0, padding='post', maxlen=review_len)
```

Exemples

Analyse des critiques de films : word embedding

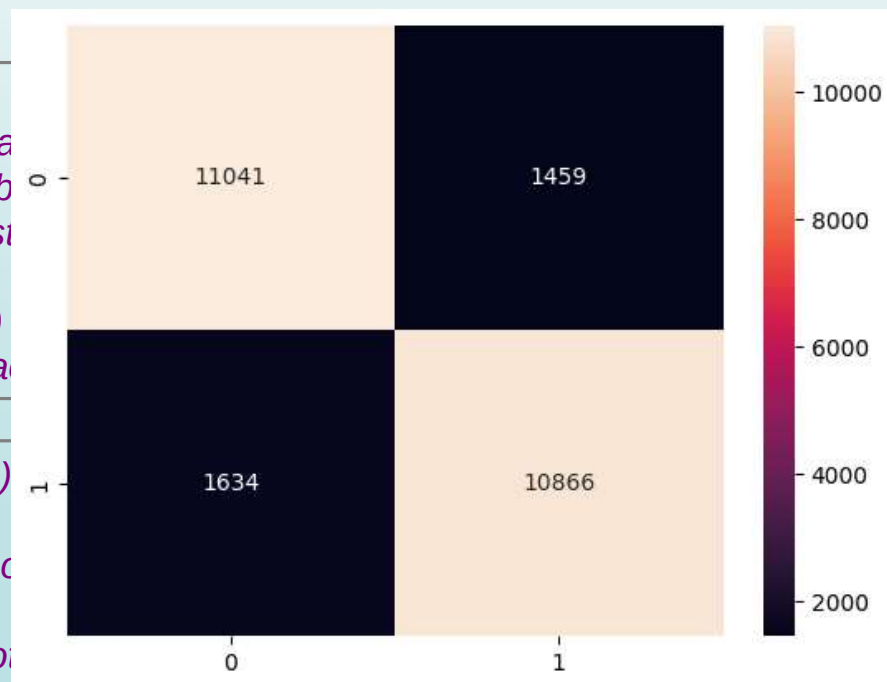
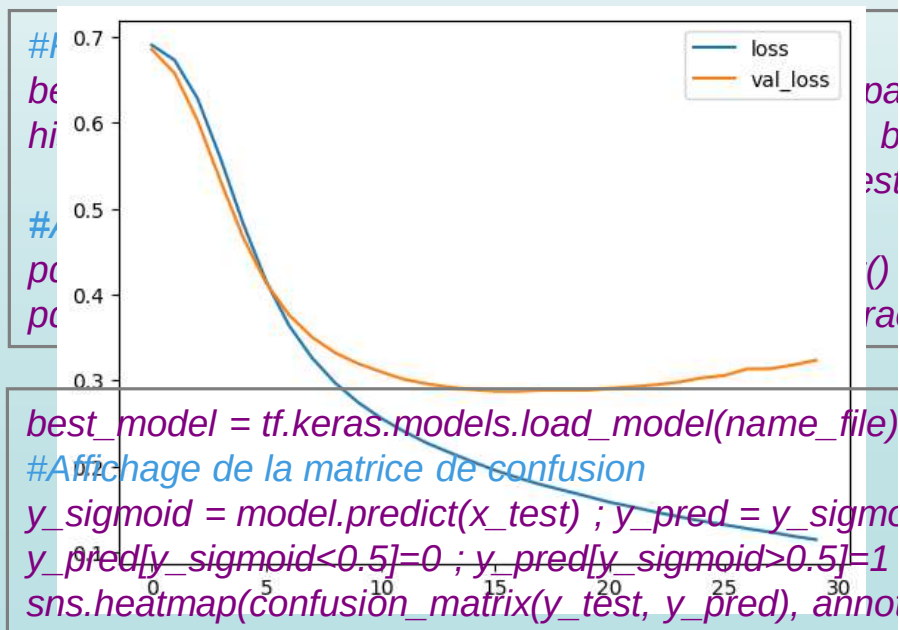
- La couche **embedding** est capable en réduisant la dimension de capturer le contexte, la similarité sémantique et syntaxique d'un mot.
- Les poids sont ajustés lors de la phase d'apprentissage, ce qui permet de sélectionner les caractéristiques les plus pertinentes des mots.
- Une couche d'embedding est configurée à partir de trois paramètres : `input_dim` : taille du vocabulaire, `output_dim` : taille des vecteur représentant les mots, `input_length` : taille des séquences d'entrée.

Model: "sequential"		
#Création d'un modèle séquentiel	Layer (type)	Output Shape Param #
<code>model = tf.keras.models.Sequential()</code>		
<code>model.add(tf.keras.layers.Input(shape=(review_len,)))</code>		
<code>model.add(tf.keras.layers.Embedding(input_dim=num_words, output_dim=16, input_length=review_len))</code>	embedding (Embedding)	(None, 256, 16) 160000
<code>model.add(tf.keras.layers.GlobalAveragePooling1D())</code>	global_average_pooling1d (GlobalAveragePooling1D)	(None, 16) 0
<code>model.add(tf.keras.layers.Dense(16, activation='relu'))</code>	dense (Dense)	(None, 16) 272
<code>model.add(tf.keras.layers.Dense(1, activation='sigmoid'))</code>	dense_1 (Dense)	(None, 1) 17
Total params: 160289 (626.13 KB)		
Trainable params: 160289 (626.13 KB)		
Non-trainable params: 0 (0.00 KB)		
# Modèle d'apprentissage	<code>model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])</code>	

Exemples

Analyse des critiques de films : apprentissage

- Les modèles d'apprentissages avec des couches d'embedding peuvent très rapidement être en sur-apprentissage.
- Afin de conserver le meilleur modèle il est possible d'utiliser une fonction de callback afin d'effectuer une action (sauvegarde) du modèle au cours des différentes epochs.

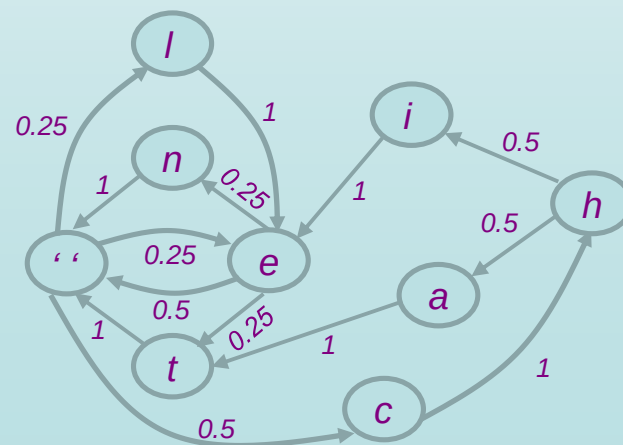


Exemples

Génération de texte par caractères : Chaîne de Markov

- Une chaîne de Markov est un outil de la théorie des processus aléatoires, constitué d'une séquence d'états.
- Les liaisons entre deux états ne sont créées que si il existe la possibilité de passer d'un état à l'autre.
- Ainsi l'information utile pour la prédiction du futur est entièrement contenue dans l'état présent.
- Le système n'a donc pas de « mémoire ».
- Pour chaque état il existe alors une probabilité de transition pour passer d'un état courant vers un état futur.

le chien et le chat



Exemples

Génération de texte par caractères : Chaine de Markov

- Création d'une chaine de markov de type dictionnaire indexée sur les états et contenant le décompte des caractères suivants de l'état.

```
from collections import Counter, defaultdict    #Dictionnaire de conteur
def chaine_markov(train_text, etat_size):
    model = defaultdict(Counter)
    for i in range(len(train_text) - etat_size):
        etat = train_text[i:i+ etat_size] ; etat_suivant = train_text[i + etat_size]
        model[etat][etat_suivant] += 1
    return model
```

- Génération d'un texte à partir d'une chaine de markov.

```
def create_texte(model, texte_size):
    continuer=True ; texte = []
    while continuer:
        etat = random.choice(list(model)) ; texte += list(etat)
        for x in range(texte_size) :
            etat_suivant = random.choices(list(model[etat]) , model[etat].values())
            texte.extend(etat_suivant) ; etat = etat[1:]+text[-1]
            if len(texte)>=text_size: continuer = False ; break
    return("".join(texte))
```

Exemples

Génération de texte par caractères : Chaîne de Markov

- L'objectif est de générer un texte à partir du roman d'Alexandre Dumas : 'Le comte de Montecristo'.
- Le texte utilisé est dépourvu de ponctuations et de majuscules.

```
corpus=open('DATA/Le comte de Monte Cristo.txt').read()
```

- Génération du texte à partir d'états composés d'un seul caractère.

```
model = chaine_markov(train_text , 1)  
print(create_texte(model, 200))
```

ît jonsone ndaquurre vit l à let daicos ureure lers sen'yeixat
viste urest ci s fe e m'a bla fantourtie uvre t

- Génération du texte à partir d'états composés de douze caractères.

```
model = chaine_markov(train_text , 12)  
print(create_texte(model, 200))
```

un jeune homme suivit des yeux et qui nous fait cette chère
enfant et reprends courage en songeant qu'il y avait en
outre deux ou trois ans à son œuvre il n'avait plus dans
l'assemblée en ce moment et

- Avec un état composé de douze lettres les mots apparaissent clairement ainsi qu'un début de sens pour les phrases.

Exemples

Génération de texte par mots : Chaîne de Markov

- Afin d'améliorer les résultats on va utiliser la bibliothèque **markovify** qui est entraînée sur un énorme corpus.
- Les états sont des mots et non des caractères, de plus le modèle est en mesure d'identifier les ponctuations.

```
import markovify
model = markovify.Text(corpus, 6) ; line=0
while line<4 :
    sent = model.make_short_sentence(max_chars=2000)
    if sent : print(sent) ; line += 1
```

Ce brave homme n'a qu'à dire seulement, que je suis le premier commis de la maison Thomson et French de Rome se présenta chez M. Morrel.

Aussi que ne ferais-je pas pour le servir ! – À la bonne heure, dit Eugénie ; parlons franc, monsieur, j'aime cela.

– C'est vrai, monsieur, dit Dantès, et j'en demande bien humblement pardon à vous, madame la baronne et à Mlle Danglars.

» Danglars voulut au moins se donner le mérite de la bonne volonté : la rétraction n'en sera pas moins faite.

« – Alors, prends mon bras et partons, car nous n'avons pas de temps à perdre, je vais vous interroger et vous me répondrez.

Exemples

Génération de texte par caractères : LSTM

- Nous utilisons un modèle composé de seulement 44 caractères.
- Les caractères accentués et certaines lettres telles que le [*<, >, (,), -*] ont été supprimées.
- Le modèle est entraîné au niveau des caractères et non des mots, afin de limiter le nombre de valeurs différentes à traiter.
- Les caractères sont encodés dans les réseaux LSTM.

```
caracteres = sorted(list(set(corpus)))  
encode = { c : i for i, c in enumerate(caracteres)}  
decode = { i : c for i, c in enumerate(caracteres)}
```

```
#Dictionnaire des caractères  
{'\n': 0, ' ': 1, '!': 2, '"': 3, ',': 4, '.': 5, '0': 6, '1': 7,  
...'u', 39: 'v', 40: 'w', 41: 'x', 42: 'y', 43: 'z'}
```

- Pour estimer le caractère suivant dans le texte les LSTM doivent mémoriser la liste des caractères qui le précède.
- Les caractères doivent donc être fournies dans l'ordre du texte sous forme de séquence.
- La taille des séquences est un des hyperparamètre du modèle.

Exemples

Génération des data

- Les entrées sont des séquences de `seq_length` (100) caractères.
- Pour chaque séquence la target est 101^{ième} caractère, c'est-à-dire le caractère qui vient juste après la séquence.

```
seq_length = 100
n_chars = len(corpus)           #200000
n_vocab = len(caractres)       #44
X = [] ; y = []
```

```
for i in range(0, n_chars - seq_length) :
    seq_in = corpus[i:i + seq_length]
    seq_out = corpus[i + seq_length]
    X.append([encode[char] for char in seq_in])
    y.append(encode[seq_out])
```

- Les séquences d'entrée doivent être sous une forme acceptable par les lstm (samples, taille des séquences, features)
- Pour faciliter l'apprentissage les données doivent être normalisées.
- Enfin les données de sortie doivent être converties en oneHot.

```
X = np.reshape(X, (len(X) , seq_length, 1))
X = X / float(n_vocab)
y = to_categorical(y)
```

#Formatage des données
#Normalisation des entrées
#Onehot encoding

Exemples

Modèle récurrent

- *Le modèle est composé de deux couches de type LSTM de 256 neurones chacune.*
- *Les fonctions d'activation sont les fonctions par défaut (tanh).*
- *Des couches de type Dropout avec une probabilité de 20% sont ajoutées après chaque couche de LSTM.*
- *La couche de sortie est de type dense avec une activation softmax pour la prédiction d'une probabilité pour chacun des 39 caractères.*

```
model = Sequential()  
model.add(LSTM(256, input_shape=(X.shape[1], X.shape[2]), return_sequences=True))  
model.add(Dropout(0.2))  
model.add(LSTM(256))  
model.add(Dropout(0.2))  
model.add(Dense(y.shape[1], activation='softmax'))
```

```
model.compile(optimizer='RMSprop', loss='categorical_crossentropy', metrics=['accuracy'])  
history=model.fit(X, y, batch_size=128, epochs=10, shuffle=True, validation_split=0.2)
```

Les transformers

- La notion d'attention
- Bloc encodage
- Bloc décodeur

Les transformers

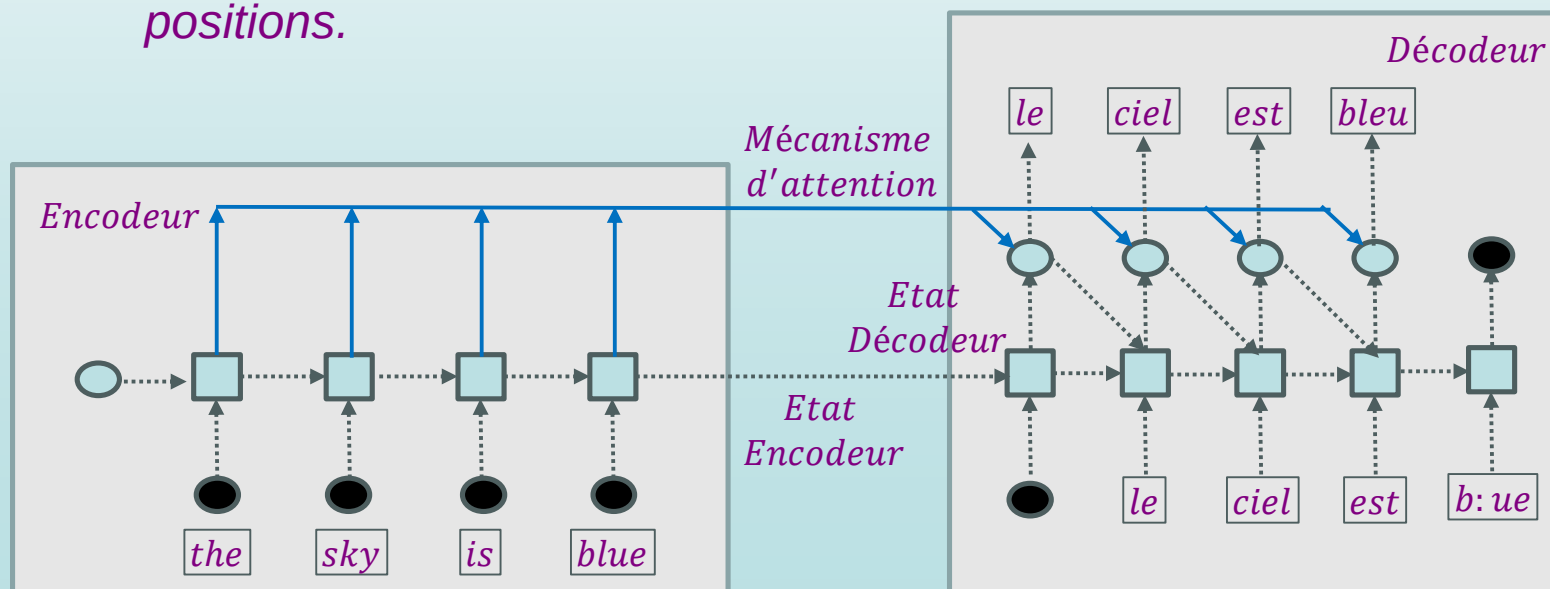
Apprentissage "plus" profond

- Le Natural Language Processing, consiste à effectuer des tâches telles que la synthèse, le résumé, la traduction, l'analyse, la reconnaissance vocale de contenus textuels ou sonores.
- Les TAL doivent comprendre les mots, mais aussi comment lorsqu'ils sont liés entre eux ils peuvent former des phrases totalement différentes.
- De part leurs structures les RNNs ne permettent pas la parallélisation lors de l'apprentissage.
- Et si les séquences sont trop longues, le modèle a tendance à oublier le contenu des mots distants ou à le mélanger avec les mots suivants.
- Les **Transformers** sont devenus l'une des solutions pour surmonter le problème lié à l'oubli dans les RNNs : "vanishing gradients".
- Ils permettent la modélisation des dépendances sans tenir compte de leur distance dans les séquences d'entrées ou de sorties.
- L'approche fondamentale est basée une notion d'**Attention** qui accorde à chaque mot du langage une importance en fonction du contexte.

Les transformers

Les encodeurs-décodeurs avec attention

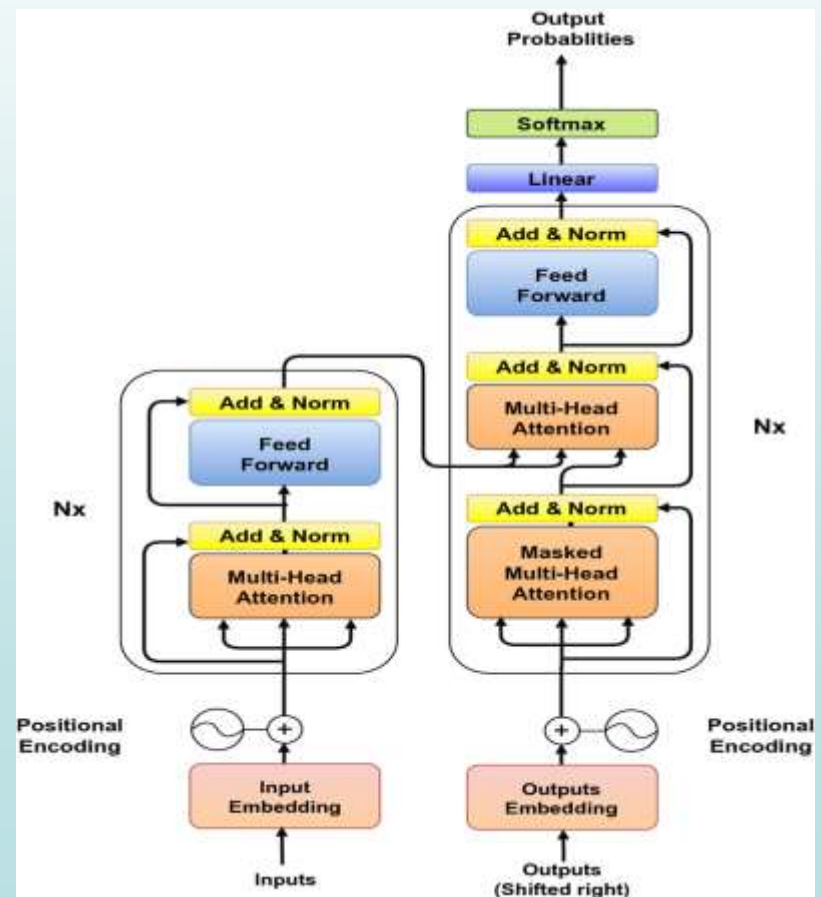
- Les Transformers et les systèmes d'attention utilisent des encodeurs et des décodeurs pour l'apprentissage profond.
- Afin de pouvoir prendre en compte les informations des tokens les plus éloignés le décodeur aura un accès direct à ces informations.
- L'encodage fournit au décodeur des informations sur la phrase dans sa globalité, ainsi que les liens entre les tokens indépendamment de leurs positions.



Les transformers

"Attention is all you need"

- Est un article de 2017, écrit par des scientifiques de google et de l'université de Toronto, qui présente l'architecture des transformers.
- Cette architecture est à la base des modèles de TNL tels que : BERT, GPT v1-4, Mistral, LLaMa, T5.
- L'architecture se décompose en de deux blocs : un bloc encodeur et un bloc décodeur.
- Chaque bloc est constitué d'une pile d'encodeurs et d'une pile de décodeurs.
- Le nombre d'encodeurs et de décodeurs sont de même taille.
- Chaque couche traite des séquences de vecteurs.



Les transformers

Couches d'embeddings

- L'étape *Input Embedding* transforme chaque mot (ou token) en un vecteur dense qui capture ses propriétés sémantiques et syntaxiques.
- L'embedding d'entrée est un vecteur correspondant à la somme de trois embedding, un sur les mots, un sur la position et un sur le segment.
- L'embedding des mots, consiste à tokenizer puis à projeter les tokens dans un espace vectoriel de dimension fixe (embedding classique).
- Contrairement aux RNN les Transformers ne traitent pas les tokens séquentiellement, aussi un embedding de position doit être ajouté.
 - *Positional Embeddings fixes* : Est une transformation sinusoïdale telle que proposée par l'article "Attention is All You Need".
 - *Positional Embeddings appris* : Les paramètres sont appris par le modèle comme pour l'embedding des mots.
- *Segment Embedding* Utilisé quand le modèle doit distinguer plusieurs phrases dans la même séquence

Les transformers

Positional encoding : PE

- Pour chaque position (pos) des mots et pour chaque dimension i du vecteur d'embedding, les coefficients du vecteur PE sont :

$$PE_{(pos,i)} = \sin\left(\frac{pos}{10000^{\frac{i}{d_m}}}\right) \text{ si } i \text{ pair} \quad PE_{(pos,i)} = \cos\left(\frac{pos}{10000^{\frac{i-1}{d_m}}}\right) \text{ sinon}$$

$pos = [0..6]$
 $d_m = 9$

pos	0	1	2	3	4	5	6	7	8	9
0	0,000	1,000	0,000	1,000	0,000	1,000	0,000	1,000	0,000	1,000
1	0,841	0,540	0,129	0,992	0,017	1,000	0,002	1,000	0,000	1,000
2	0,909	-0,416	0,255	0,967	0,033	0,999	0,004	1,000	0,001	1,000
3	0,141	-0,990	0,378	0,926	0,050	0,999	0,006	1,000	0,001	1,000
4	-0,757	-0,654	0,494	0,869	0,067	0,998	0,009	1,000	0,001	1,000
5	-0,959	0,284	0,602	0,799	0,083	0,997	0,011	1,000	0,001	1,000
6	-0,279	0,960	0,700	0,714	0,100	0,995	0,013	1,000	0,002	1,000

embedding PE
mot en position 2

- Cette technique permet de calculer les distances relatives, les positions proches ont des vecteurs similaires.
- Le modèle n'a pas de position à apprendre : elle ne dépend pas des données d'entraînement
- PE permet également d'extrapoler sur des séquences longues.

Les transformers

Couches d'embeddings selon les modèles

- Les modèles BERT utilisent des embeddings de mots, un Positional Embeddings appris ainsi qu'un embedding de segment.
- Les modèles GPT (2-4) n'utilisent pas d'embedding de segment.
- Ils ont la particularité d'être causal : les modèles sont en mesure de prédire les prochains mots en fonction uniquement des mots précédents.
- Certains modèles simples (Transformer-XL, T5...) utilisent le PE fixes.
- Les modèles modernes tels LLaMa ou Mistral utilisent eux une version améliorée des sinusoides : RoPE (Rotary Positional Embedding).
- A chaque couple (x_{2i}, x_{2i+1}) de l'embedding des mots on applique une rotation d'un angle $\theta_i = 10000^{-2i/d_m}$ en fonction de la position p du mot :

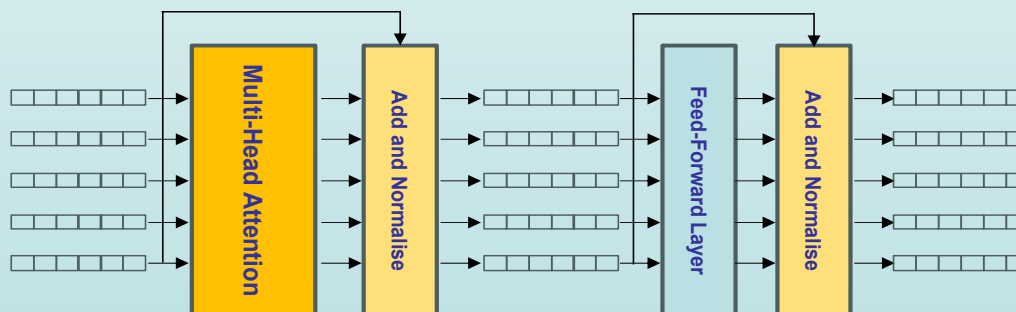
$$\text{RoPE}(x_p) = (x_{2i} \cos(p\theta_i) - x_{2i+1} \sin(p\theta_i), x_{2i} \sin(p\theta_i) + x_{2i+1} \cos(p\theta_i))$$

- RoPE conserve la norme (embedding des mots), donc ne modifie pas sa représentation contrairement au PE fixe.
- Le produit scalaire permet de d'accéder aux différences de positions.

Les transformers

Bloc d'encodage

- L'encodeur reçoit une séquence d'entrée et la transforme en une suite de vecteurs contenant des informations contextuelles sur les mots.
- Les séquences d'entrée sont des tokens correspondant au minimum à un embedding des mots et de position.
- Ces séquences sont normalisées et on y applique un dropout.
- L'encodeur est un empilement de N couches identiques composé de deux sous blocs principaux (12 pour BERT-base, 24 pour BERT-large).



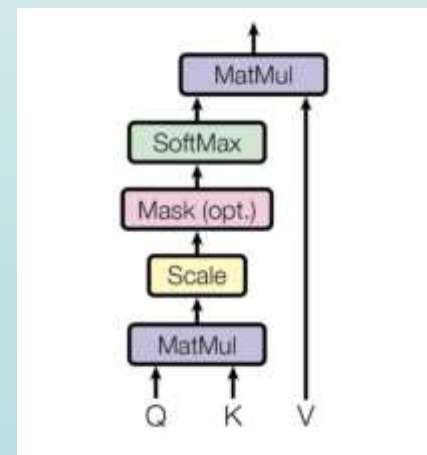
Les transformers

Scaled Dot-Product Attention

- C'est à l'intérieur des Multi-Head Attention, composés de "Scaled Dot-Product Attention" que toute l'innovation est concentrée.
- Pour chaque mot on construit une représentation contextuelle en fonction des autres mots.
- Le produit QK^T génère un "Attention Matrix Scores", cette matrice permet de connaître "la corrélation" entre chaque token.
- Les matrices Q et K ont une taille $d_k = d_m / h$, où h est le nombre de têtes d'attention. Pour chaque tête le calcul le Scaled Dot-Product Attention

$$\text{Head}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) * V$$

- QK^T mesure la similarité entre les mots.
- $\sqrt{d_k}$ permet de normaliser les valeurs pour éviter l'évanouissement du gradient.
- softmax pondère les mots selon leur importance

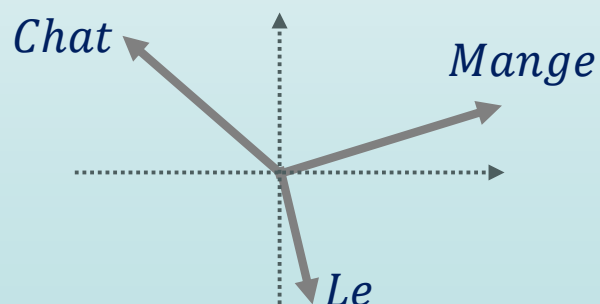


Les transformers

Scaled Dot-Product Attention

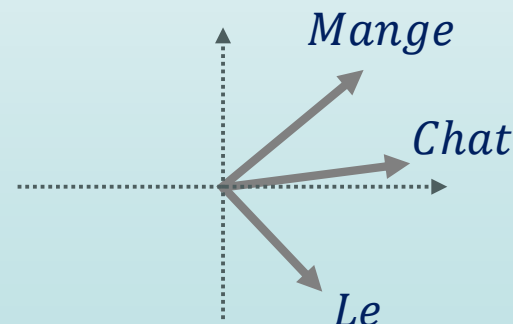
- Le résultat consiste à transformer la représentation des tokens issues de "l'embedding" a une représentation dans un nouvel espace latent.
- La représentation consiste à décrire chaque tokens en fonction des relations qui les lient.
- Deux tokens liés auront alors des vecteurs les représentants proches.

Le chat mange



L'embedding encode les mots sans prendre tenir compte d'un contexte particulier

Scaled Dot-Product Attention

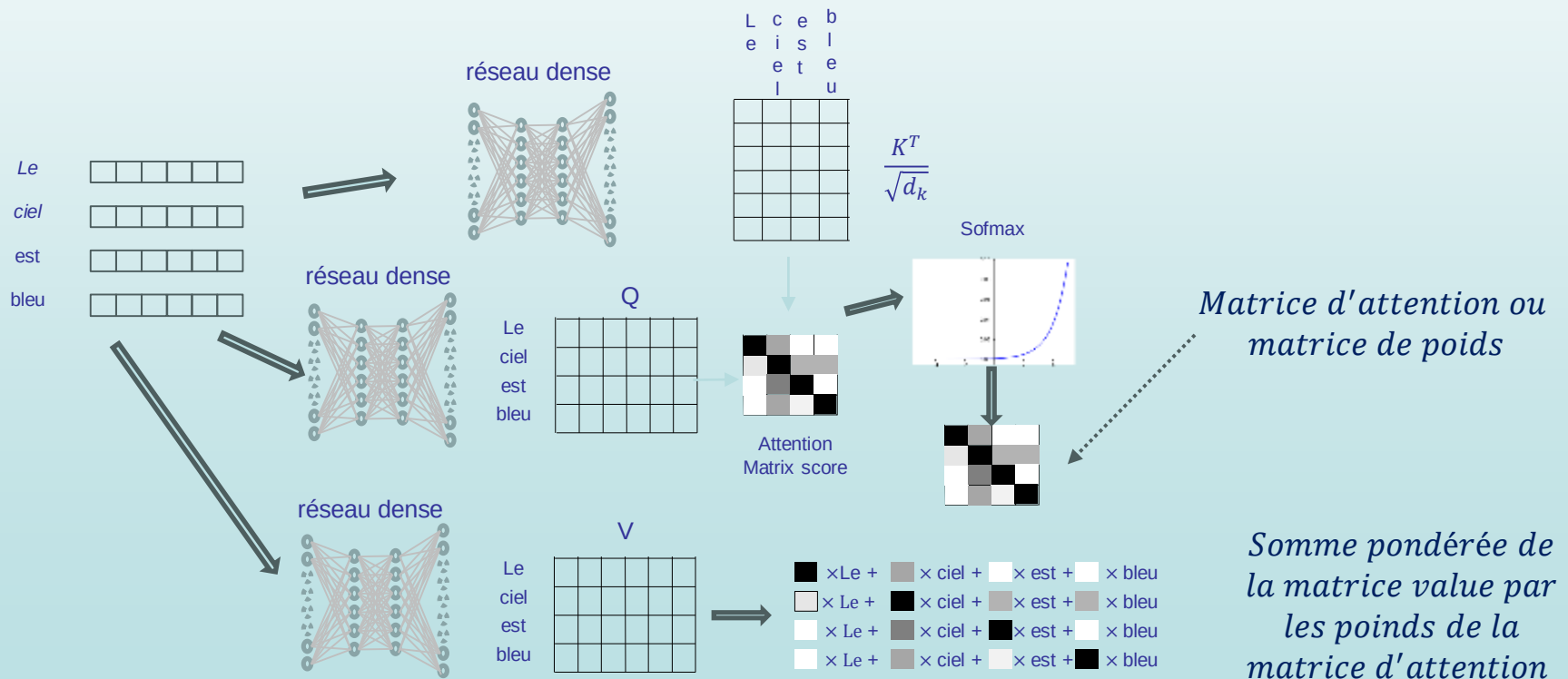


La représentation des mots est liée au contexte

Les transformers

Scaled Dot-Product Attention

- Dans les encodeurs il n'y a pas de couche "Mask". Cette couche permet de limiter l'attention aux mots qui précèdent.
- Dans l'encodeur un token est lié à tous les mots du contexte.



Les transformers

Scaled Dot-Product Attention

```
# Création d'un dictionnaire d'embedding
embeddings = { 'Le': np.array([0.1, 0.2, 0.3]), 'ciel': np.array([0.4, 0.5, 0.6]),
                'est': np.array([0.7, 0.8, 0.9]), 'bleu': np.array([1.0, 1.1, 1.2])}
sentence = list(embedding.keys()) # Création d'une phrase
embedded_tokens = np.array(list(embeddings.values())) # Vectorisation de la phrase
```

```
# Initialisation des matrices query (Q), clés (K) et valeurs (V) identiques aux embeddings
Q = K = V = tf.constant(embedded_tokens, dtype=tf.float32)
```

```
# Définition de la fonction d'attention
def scaled_dot_product_attention(q, k, v, mask=None):
    QK = tf.matmul(q, k, transpose_b=True) # Calcul de la similarité entre mot i (Q) et j (K)
    similarite = QK / tf.cast(tf.shape(k)[-1], tf.float32) # Normalisation
    # Le mask permet d'ignorer certains éléments (évite de regarder les mots futurs)
    if mask is not None:
        similarite += (mask * -1e9) # Les positions masquées sont presque nulles ( $\approx -\infty$ ).
    # softmax transforme les scores de similarité en probabilités
    weights = tf.nn.softmax(similarite, axis=-1)
    output = tf.matmul(weights, v) # Calcul de la sortie pondérée
    return output, weights
```

Les transformers

Multi-head Attention

- "Multi-head Attention" fait référence à la capacité du mécanisme de se concentrer simultanément sur plusieurs "aspects" d'un texte.
- Avec un seul modèle de type Scaled Dot-Product Attention le mécanisme va finalement se concentrer sur un seul aspect.

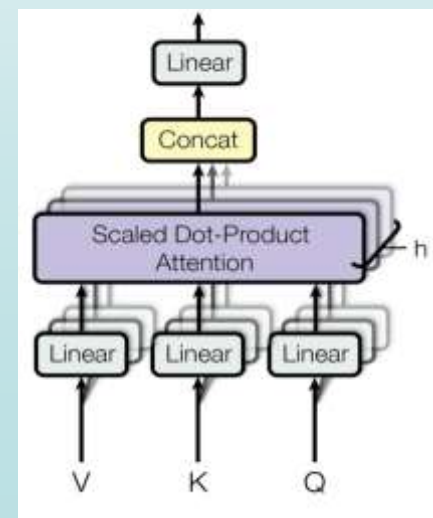
Hier j'ai acheté une veste dans un magasin de haute couture

Si l'on s'intéresse à l'objet c'est la veste qui aura un poids plus important

Si notre intérêt se porte le temps le mot hier sera privilégié

On peut être intéressé par l'action ou la qualité du produit

- Si l'attention se porte sur un aspect on perd de l'information sur les autres.
- Pour avoir des informations sur les différents aspects on utilise plusieurs blocs d'attention (tête d'attention).
- Les résultats sont concaténés afin d'obtenir une information plus complète sur le texte.



Les transformers

Multi-head attention

- Le Scaled Dot-Product Attention est répété par chaque tête d'attention.
- Ces opérations sont effectuées en parallèle.
- Chaque tête se concentre sur différents aspects linguistiques.
- La représentation finale correspond à la concaténation des différentes têtes d'attentions suivit d'une projection linéaire.

$$\text{MultiHead}(X) = \text{concat}(\text{Head}_1(Q_1, K_1, V_1), \dots, \text{Head}_n(Q_n, K_n, V_n)) * W_0$$

- Chaque matrice est obtenue à partir de matrices de poids W apprises ($Q_1 = XW_{Q1}, K_1 = XW_{K1}, V_1 = XW_{V1}$)
- Les vecteurs d'embeddings d'entrée sont combinés avec les résultats des mutl-head attention puis normalisés

$$Y = \text{Normalise}(X + \text{MultiHead}(X))$$

- La normalisation permet de mieux traiter les gradients lors de l'algorithme de rétro-propagation, et évite son évanouissement.

Les transformers

Exemple :

- Pour une séquence d'entrée X composée de n tokens, pour un encodage d_m et h têtes d'attention.
- La dimension des matrices d'attention est donc $d_k = d_m/h$.
- La couche d'entrée reçoit des matrices $X \in \mathbb{R}^{n \times d_m}$
- Les poids des tête d'attention i sont : $(W_{Qi}, W_{Ki}, W_{Vi}) \in \mathbb{R}^{d_m \times d_k}$
- Ainsi $(Q_i, K_i, V_i) \in \mathbb{R}^{n \times d_k}$
- Les dimensions des "head" sont donc

$$\dim(\text{Head}(Q_i, K_i, V_i)) = (n \times d_k)(d_k \times n) * \dim(n \times$$

- La concaténation de h tête conduit à une matrice $n \times (h \times d_k)$
- La matrice W_0 permettant la projection de la superposition des h tête d'attention est donc $(h \times d_k) \times d_m$

$$\dim(\text{MultiHead}(Q, K, V)) = \dim(n \times (h \times d_k)) * \dim((h \times d_k) \times d_m) = \dim(n \times d_m)$$

- La sortie du multi-head attention a la même dimension que l'entrée X .

Les transformers

Feed-Foward Layer

- Le sous-bloc de self-attention est linéaire, il consiste simplement à agréger différentes représentations des mots : $Head(Q, K, V) = coef * V$
- Le softmax introduit une non linéarité, mais elle agit seulement sur les relations entre les tokens (coef), et pas sur la représentation finale.
- FFL est un réseau entièrement connecté qui introduit la non linéarité.

$$FFL(Y) = Relu(YW_1 + b_1)W_2 + b_2$$

- $W_1 \in \mathbb{R}^{d_m \times d_{ff}}$, $b_1 \in \mathbb{R}^{d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times d_m}$, $b_2 \in \mathbb{R}^{d_m}$ Avec d_{ff} souvent quatre fois plus grand que d_m .
- Le self-attention permet de savoir à quels mots sont liés aux tokens, le FFL permet de savoir comment traiter ces informations.
- Un FFL à deux couches est un approximateur universel : il peut approcher n'importe quelle fonction continue.
- Le FFL est ensuite combiné avec un layer de normalisation

$$Z = Normalise(Y + FFN(Y))$$

Les transformers

Modèle BERT

- *BERT (Birectional Encoder Representation from Transformers) est le premier modèle basé sur les transformers, développé par Google AI.*
- *L'objectif est de comprendre le sens des mots dans leur contexte, grâce à une lecture bidirectionnelle du texte.*
- *Ce modèle est uniquement composé d'une partie encodeur.*
- *L'embeddings correspond à la somme de trois vecteurs : un embedding de mot, un embedding de position et un embeddings de segment.*
- *Deux tâches permettent de pré-entraîner le modèle*
 - *Masked Language Modeling (Masked LM) consiste à prédire les mots manquants (environ 15%).*
 - *Next Sentence Prediction (Causal LM) consiste à prédire la phrase B qui suit la phrase A, à partir d'un modèle supervisé avec 50% de bonnes phrases B et 50% de phrases aléatoires.*
- *BERT peut être ensuite être ré-entraîné sur des tâches comme l'analyse de sentiments, Question-answering ou le NER avec peu d'époques.*

Les transformers

Bloc de décodage

- Le décodeur reçoit deux types d'informations, les sorties de l'encodeur qui correspondent au contexte, et les tokens déjà générés.
- Un décodeur est un modèle unidirectionnel, utilisé pour la génération de texte par les modèles de type GPT.
- A un instant T le décodeur a donc accès à toutes les informations d'entrées, et seulement à celles des sorties calculées au temps $T' \leq T$.
- Lors de la phase d'apprentissage les décodeurs sont entraînés sur des séquences cibles qui contiennent des tokens aux temps $T' > T$.
- Pour ne pas que lors de l'apprentissage le décodeur soit influencé par les mots futurs présents sur les séquences cibles.
- Le Mask Multi-Head Attention utilise une matrice triangulaire inférieure M .

$$\text{Head}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) * V$$

Les transformers

Cross-Attention (Encoder–Decoder Attention)

- Dans ce bloc chaque token peut rechercher l'information pertinente dans les représentations produites par l'encodeur.
- A la sortie du bloc "Mask Multi-head Attention" on obtient des tokens

$$Y_{mask} = \text{Normalise}(X_{dec} + \text{MultiHead}(X_{dec}))$$

- Les matrices (Q_i, K_i, V_i) des têtes d'attention du second bloc Multi-Head Attention sont obtenues de la manière suivante :

$$Q_i = Y_{mask} W_{Q_i}^{dec} ; K_i = Z W_{K_i}^{enc} ; V_i = Z W_{V_i}^{enc}$$

- Dans ce bloc les clés K et valeurs V viennent de l'encodeur, et les requêtes Q viennent du décodeur.

Projection dans le vocabulaire

- La sortie des blocs de Cross-Attention les tokens sont projetés vers le vocabulaire correspondant à une projection linéaire + un softmax.

$$P(\text{mot}_{t+1}) = \text{softmax}(Z_{dec} W_{vocab} + b_{vocab})$$

Les transformers

Exemple :

- Les séquences sources X sont composées de n tokens.
- Les séquences cibles X_{dec} sont composées de m tokens
- d_m est la taille des encodages, h le nombre de têtes d'attention, $d_k = d_m/h$ celle des têtes d'attention et nv la taille du vocabulaire.
- En entrée du décodeur les matrices : $X \in \mathbb{R}^{m \times d_m}$
- La sortie du décodeur : $X_{dec} \in \mathbb{R}^{n \times d_m}$
- Le bloc du Masked Multi-Head Self-Attention $Y_{mask} \in \mathbb{R}^{m \times d_m}$
- Pour tête d'attention du bloc Cross-Attention $Q_i \in \mathbb{R}^{m \times d_m}; K_i \in \mathbb{R}^{n \times d_m}; V_i \in \mathbb{R}^{n \times d_m}$
- La sortie de ce bloc : $Z_{dec} \in \mathbb{R}^{m \times d_m}$. Ces tokens sont ensuite injecté dans le vocabulaire afin d'obtenir la probabilité associée à chaque mot.
- Une matrice : $W_{vocab} \in \mathbb{R}^{nv \times d_m}$ et $b_{vocab} \in \mathbb{R}^{m \times nv}$
- Chaque ligne correspond à la distribution de probabilité sur les nv mots possibles et ce pour chaque position de sortie.

Les transformers

Les principaux modèles

- Les modèles (GPT, LLaMA, Mistral) sont basés sur l'architecture des Transformer et génèrent du texte token par token.
- GPT 2-4 (open AI) sont des modèles très lourds avec énormément de paramètres (1.5B, 175B, 1T)
- LLaMa (meta) variant (7B, 13B, 30B, 65B, 70B), utilisent l'embedding de positionnement RoPE.
- La version 7B peut-être utilisée sur PC avec 24GO, ou 48GO pour 13B.
- Mistral (Mistral AI) variant (7B, à 8×7B).
- Chaque paramètre correspond généralement à un float sur 16 ou 32 bits. Ainsi un modèle 7B avec des float16 (2 octets) occupe 14GO.
- Afin de pouvoir utiliser les modèles avec de ressources limités (GPU, CPU, Mac Mx) il est possible de réduire la taille des paramètres.
- La quantification permet de réduire la taille des paramètres, et d'accélérer les calculs.

Les transformers

Quantification

- Généralement les poids des modèles sont stockés en float32 (FP32 – 4 octets) ou en float16 (2 octets) pour de très bonnes performances.
- La quantification consiste à enregistrer les poids au format int8 (1 octet) avec des performances qui restent bonnes.
- Il est même possible d'utiliser un format int4 (0.5 octet) mais alors les performances peuvent être moyenne.
- Les modèles open source (LLaMa, Mistral) peuvent être quantifiés localement, ce qui n'est pas le cas des modèles GPT.
- Les bibliothèques bitssandbytes (pyTorch/ Hugging Face) ; AutoGPTQ (pour GPU - CUDA) ; llama.cpp (CPU-optimisé, multi-thread) ou mlx-lm (Apple Silicon) permettent une quantification au sein du programme.
- La quantification peut conduire à une perte de précision, des problèmes de stabilité et rendre le Fine-tuning plus difficile.

Les transformers

Mise en œuvre

- *Hugging face transformers* : sont des Frameworks, multi-plateformes (CPU, GPU, TPU), open-source compatible PyTorch et TensorFlow.
- *Apple MLX* : librairie optimisée pour Apple Silicon utilisable sans GPU.

```
# Importation d'un modèle de génération de texte (GPT ou Mistral), et
from transformers import AutoModelForCausalLM, AutoTokenizer
model_name = "mistralai/Mistral-7B-Instruct-v0.2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
# Téléchargement d'un modèle de prédiction de phrases, en demi-précision
model = AutoModelForCausalLM.from_pretrained(model_name, torch_dtype=torch.float16,
                                              device_map="auto")
# Tokenisation du texte à traiter, au format torch
inputs = tokenizer_tr("Question ...", return_tensors="pt").to(model.device)
with torch.no_grad():                                # Désactivation du calcul du gradient
    outputs = model_tr.generate(**inputs, max_new_tokens=200, temperature=0.7,
                                top_p=0.9, do_sample=True)
response = tokenizer.decode(outputs[0], skip_special_tokens=True) # Décodage de la réponse
```

```
from mlx_lm import load, generate
model, tokenizer = load("mistralai/Mistral-7B-Instruct-v0.2")
print(generate(model, tokenizer, prompt))
```